

controlled collaboration throughout the deployment process.

Why GitOps for AI workloads?

Unlike conventional computer programs, managing AI workloads is more complex because their performance is influenced not only by software but also by datasets, configuration of models, training flows, and computing environment. A tiny change in data preprocessing, model formulation, or library dependencies can lead to a totally different result. Manual work with these components makes it impossible to replicate a previous experiment or deployment.

GitOps solves this problem by using version-controlled repositories to store configuration files of the AI services. With the help of Git, teams track model configurations, definitions of workflows, versions of containers, deployment instructions and requirements for infrastructure. Everything is recorded in commits and pull requests, which helps track the process better.

Moreover, GitOps ensures uniformity in the development, testing and production stages. For example, a model tested with specific dependencies and resources can be deployed using the same configuration. Thus, problems caused by the difference in environments are eliminated. Once the updated model has been developed, an automated scheme can check its performance, modify deployment configuration, and deploy it.

Another important advantage is controlled model rollback. If a newly deployed model produces inaccurate predictions, excessive latency or unexpected behaviour, the team can restore a previously stable configuration from Git. GitOps controllers also detect unauthorised or accidental changes in the live environment and reconcile it with the approved state.

Therefore, GitOps provides the traceability, reproducibility and automation needed for reliable

AI operations. It connects model development with deployment management and helps different teams manage complex AI systems through a common, auditable workflow.

Infrastructure as Code for AI

Infrastructure as Code, or IaC, entails defining and provisioning computing infrastructure via configuration files as opposed to executing it manually. AI applications generally need specialised resources like GPUs or technical infrastructure comprising high-performance storage, container clusters, and scalable computing environments. Manually configuring these resources tends to be tedious and results in inconsistent environments. However, IaC leads to repeatable, versioned, and relatively easy-to-manage infrastructure.

For instance, tools such as Terraform and OpenTofu can describe cloud resources like virtual machines, networks, storage resources, and Kubernetes clusters, while Kubernetes manifests, as well as Helm charts, can specify resources for training ML models, including container image URLs, and CPU and GPU limitations. Such configuration files can be stored in Git for future review before any infrastructure changes are performed.

An AI project may necessitate distinct environments for experimentation, model testing, and real-world deployment. Infrastructure as Code (IaC) allows teams to establish these environments using uniform reusable templates while modifying only certain parameters, thus minimising configuration discrepancies and ensuring consistent behaviour of the model.

The combination of IaC with GitOps means that infrastructure amendments follow a planned process. One member of the team proposes a change by means of a pull request, and the configuration in question is verified automatically. Approved modifications are automatically made in the environment. Previous versions are preserved in Git,

which makes revoking changes possible.

So, by regarding computing resources and deployment environments as versioned software components, IaC provides a reliable and replicable base for AI systems.

GitOps workflow for AI pipelines

A GitOps workflow for AI pipelines links the process of developing models, conducting automated validation, and deploying the models through repositories under version control. A data scientist starts the process by changing the preprocessing code, the model structure, the parameters of the training, or the configuration of the pipeline. Rather than making the changes straight away in the production setting, the new files are registered in a new branch in Git, and a pull request is created.

The creation of a pull request instigates a process of continuous integration, which involves carrying out all kinds of automated verifications. These checks may consist of code quality checks, validation of data schema, checks for dependencies, unit tests, etc. Once initial checks are completed successfully, the pipeline trains the model based on the approved version of the dataset and creates logs with key training characteristics, such as parameters of the model, duration of the training, quality metrics, and the artifacts generated.

The assessed model is then put to the test according to preset criteria of acceptance. For instance, the system could necessitate minimum accuracy and F1-scores, tolerated inference time, and no considerable decrease in performance compared to the model that is currently used. If the model does not satisfy any required condition, it will be denied, and the development team will receive recommendations for improvement. Otherwise, the successful model will be stored in a model registry with its deployment being updated in Git.